

Inside the LLM: Prediction, Context and Agents

1. How LLMs "Think" (The Mechanics of Chain of Thought)

LLMs are not biological brains; they are prediction engines.

At their core, Large Language Models (LLMs) only do one thing: **Next Token Prediction**. They calculate the statistical probability of the next word based on the words that came before.

So, when a model says it is "*Thinking*" or "*Reasoning*," it is simply **predicting a conversation with itself** before showing you the final answer.

2. The "Scratchpad" Mechanism

Since an LLM cannot "pause" to think silently, it must write to think.

- **Standard LLM:** Input Immediate Answer Prediction.
- **Reasoning LLM:** Input **[Hidden Internal Monologue]** Final Answer.

The "Thinking" block is effectively a **Scratchpad**. By generating text in this scratchpad, the model manipulates its own **Context Window** to increase the accuracy of the final answer.

Note :- **Scratchpad:** A conceptual term for the **intermediate tokens** generated by an LLM during a "Chain of Thought." It allows the model to externalize its reasoning, effectively creating a history of its own logic that it can "read" to ensure the final prediction is accurate.

3.1 Definition A: Generating Intermediate Steps (Computation)

Corresponds to: "Breakdown into phases"

Complex problems (like math or coding) cannot be solved in a single "guess." The model needs to perform computation. Since LLMs compute by generating tokens, **more tokens = more computation**.

How it works:

1. **The Constraint:** An LLM cannot look ahead; it can only look back.
2. **The Process:** By writing out Step 1, the model adds Step 1 to its memory (Context).

3. **The Result:** When it tries to predict Step 2, it is no longer looking just at your question. It is looking at Your Question + Step 1.

Insight: "Thinking" transforms a hard logic problem into a simple pattern-matching task. It is statistically easier to predict the number 4 if the previous words are $2 + 2 =$, rather than just Calculate the sum.

Example Log:

`User: "How many Rs are in the word Strawberry?"

Without Thinking: "There are two Rs." (Hallucination due to rush)

With Thinking (Hidden):

- "I need to look at the word S-t-r-a-w-b-e-r-r-y."
- "Let's count them."
- "1st 'r' is at index 3."
- "2nd 'r' is at index 8."
- "3rd 'r' is at index 9."
- "Total count is 3."

Final Output: "There are 3 Rs."

2. Definition B: Framing the Question Well (Clarification)

Sometimes the user's prompt is vague, ambiguous, or a "trick question." The thinking phase allows the model to "re-prompt" itself. It acts as a self-correction loop where the model defines the parameters before attempting to solve.

How it works:

1. The model analyzes the user's prompt.
2. It predicts text that critiques or expands on the prompt.
3. It explicitly states assumptions or definitions in the hidden text to guide its future generation.

Insight: This is essentially Self-Prompting. The model writes a better, more detailed version of your question into its own memory so the final answer is higher quality.

Example Log

`User: "Write a short story about a bank."

Thinking (Hidden):

- "Wait, 'bank' is ambiguous."
- "Does the user mean a financial bank or a river bank?"
- "Given the context of previous questions, they likely mean a river bank."

- "I should focus on nature and flow, not money and vaults."

Final Output: "The water flowed gently against the grassy bank..."`

3. Visualizing the "Thinking" Process

Here is how the "Thinking" tokens turn into "Answer" tokens.

Scenario: The model is solving a math problem.

Step	Current Input (Context)	Model Predicts (ID)	Tokenizer Converts To	User Sees
1	[User Question]	ID: 991	"First,"	(Thinking...)
2	[User Question] + [991]	ID: 32	"I"	(Thinking...)
3	... + [991, 32]	ID: 450	"calculate"	(Thinking...)
...	(100 steps of thinking)	...	...	...
101	[Question] + [Whole Scratchpad]	ID: 8821	"The"	The
102	... + [8821]	ID: 122	"answer"	answer

4. Direct Answer to Your Question

You asked: "Model generates intermediate tokens... which goes to tokenizer... and converts to text... does this happen?"

YES.

Every single step—whether it is a "Thinking" step or a "Final Answer" step—follows this exact path:

1. Model predicts **ID**.
2. ID goes to **Tokenizer**.
3. Tokenizer turns ID into **Text**.
4. Text is appended to the **History** (so the model remembers it).
5. Text is shown (or hidden) on the **Screen**.

Notion Note Summary:

The Model acts as a **Number Generator**.

The Tokenizer acts as a **Dictionary**.

The "Thinking" process is just the model generating a long list of numbers that translate into reasoning steps, which function as context for the final numbers (the answer).

Question: So let's say original question has 15 tokens and it does thinking and generated 50 tokens more and then the final ans has 70 tokens so instead of only 15 + 70 tokens the intermediate thinking process has accumulated tokens ?

The "Snowball Effect" of Context

1. The Math

You asked: *"Instead of only 15 + 70 tokens, does the intermediate thinking process accumulate?"*

YES.

The model does not just see the Prompt (15) and the Answer (70). It treats the "Thinking" tokens exactly the same as any other text.

The Total Context at the end of the generation is:

15 (Prompt)+50 (Thinking)+70 (Answer)=135 Total Tokens
15 (Prompt)+50 (Thinking)+70 (Answer)=135 Total Tokens

2. Step-by-Step Context Growth

Here is what the model "sees" at each stage. Remember, the model is **stateless**—it forgets everything instantly unless that information is re-fed into it as input.

- **Step 0 (Start):**
 - Model sees: [15 Prompt Tokens]
 - *Task: Start thinking.*
- **Step 1 (During Thinking):**
 - Let's say it has generated 25 thinking tokens.
 - Model sees: [15 Prompt Tokens] + [25 Thinking Tokens]
 - *Task: Generate thinking token #26.*
- **Step 2 (Start of Answer):**
 - The model finishes thinking (50 tokens). Now it must generate the first word of the final answer.
 - Model sees: [15 Prompt Tokens] + [50 Thinking Tokens]
 - *Task: Generate Answer Token #1.*
 - **Crucial Point:** If we deleted the 50 thinking tokens here, the model would lose its "logic." It would go back to being dumb. It **needs** those 50 tokens in its history to know *why* the answer is what it is.
- **Step 3 (End of Answer):**
 - Model sees: [15 Prompt] + [50 Thinking] + [69 Answer Tokens]
 - *Task: Generate Answer Token #70.*

3. Why This Matters (The Cost of Thinking)

Since the "Thinking" tokens accumulate in the context:

1. **Memory Usage (Context Window):**
2. The "Thinking" tokens take up space. If a model has a limit of 8,000 tokens, and it "thinks" for 2,000 tokens, you now only have 6,000 tokens left for the actual conversation.
3. **Latency (Speed):**
4. Generating those 50 intermediate tokens takes time. You have to wait for the model to generate 15 + 50 before you see the first word of the answer.
5. **Compute Cost:**
6. In paid APIs (like OpenAI's API), you usually pay for **Input + Output**.
 - You pay for the 15 prompt tokens.
 - **You pay for the 50 thinking tokens** (because the model generated them).
 - You pay for the 70 answer tokens.

Summary for Notion

The Accumulation Rule:

Intermediate "Thinking" tokens are real tokens. They are appended to the conversation history just like the user's prompt. The model **must** "read" its own thinking to generate the final answer.

Total Load = Prompt + Thinking + Answer.

Question: Also as we know the context window is the size model can remember at once and generally a thumb rule is that 750 words = 1000 tokens so if there would be only input and output token instead of these thinking ones the context window would have been larger and it is due to this thing that after long chat reached in chatgpt or any other model that the model forgets the initial conversation?

The Cost of Thinking: Context Window Erosion

1. The Finite Bucket

The Context Window is like a bucket with a fixed size.

- **Rule of Thumb:** ~750 words = 1,000 tokens.
- **The Limit:** If a model has a 128k context window, it can hold roughly 128,000 tokens total.

The Formula for a Chat Session:

Space Used=Past Chat History+Current Prompt+Thinking Tokens+Final Answer
Space Used=Past Chat History+Current Prompt+Thinking Tokens+Final Answer

2. The "Sliding Window" Effect

You asked: *"Is it due to this that the model starts forgetting the initial conversation?"*

Yes.

When the bucket gets full, the system acts like a **Sliding Window**. To make room for new tokens (the new answer + the thinking), the system **deletes** the oldest tokens (the start of the chat).

Compare Standard vs. Reasoning Models:

Scenario: You have a Context Limit of 4,000 tokens (for simplicity).

Feature	Standard Model (e.g., GPT-4o mini)	Reasoning Model (e.g., o1)
User Prompt	50 tokens	50 tokens
Thinking	0 tokens	1,000 tokens (Hidden scratchpad)
Final Answer	100 tokens	100 tokens
Total per Turn	150 tokens	1,150 tokens
Result	You can have ~26 turns before the memory fills up.	You can only have ~3 turns before the memory fills up.

The Consequence:

Because the "Thinking" model generates thousands of invisible tokens to be smart, it burns through your Context Window 10x faster. It will "forget" your name or the first prompt much sooner than a standard model would.

3. Visual Analogy: The Whiteboard

Imagine a classroom whiteboard (The Context Window).

- **Standard Model:** The teacher writes the answer neatly in the corner. There is plenty of room left on the board for the history of the class.
- **Thinking Model:** The teacher fills the **entire board** with rough calculations, formulas, and scratch work to get the answer right.
 - **The Problem:** To write the *next* question, the teacher has to **erase** what was written at the top of the board (the beginning of the class).

4. Technical Note: "Hidden" doesn't mean "Free"

Even though the interface (like ChatGPT) might hide the thinking tokens from you (collapsing them into a "Thought" icon), **they still exist in the GPU memory.**

1. The model generated them.
2. The model had to "read" them to produce the final answer.
3. Therefore, they occupy the Context Window just as much as visible text.

Summary for Notion

Context Eviction:

"Thinking" is computationally expensive in terms of **memory space**.

Since every reasoning step adds tokens to the context, "Thinking Models" consume the Context Window significantly faster. This forces the system to truncate (cut off) the beginning of the conversation earlier than usual, leading to faster "amnesia" regarding the initial prompts.

Notes:- Fresh Chat = Empty Array

Starting a new chat disconnects the model from the previous Session ID. The application stops sending the old logs. The token counter resets to (almost) zero, freeing up the entire Context Window for a new topic.

Question: Since LLM only predicts than how it tells the current info like scorecard of a match, and why while giving prompts when it is asked to tell weather without using external resources it fails to do so?

The Difference: Base Model vs. AI System

1. The Contradiction

- **The Rule:** You are right. A base LLM (like a file stored on a hard drive) is **frozen**. It only knows what it learned during training (e.g., up to 2023). It **cannot** predict the future or know the cricket score *right now*.
- **The Reality:** Yet, when you ask ChatGPT for the score, it gives it to you.

How?

The LLM is not answering from its **Memory**. It is answering from **Reading**.

2. The Solution: "Tool Use" (or Function Calling)

When an LLM shows you the weather or cricket scores, it is using a "Cheat Sheet."

The Process (The "RAG" or Browsing Loop):

1. **You ask:** "What is the cricket score right now?"

2. **The LLM (Prediction):** It realizes it doesn't know. So, instead of predicting an answer, it predicts a **Command**.
 - *Model predicts:* <search_google query="current cricket score India vs Australia">
3. **The Stop:** The generation pauses.
4. **The System (The App):** The application (ChatGPT/Google) sees this command. It secretly goes to the internet, searches Google, and copies the text from the top result (e.g., "Ind: 240/3").
5. **The Injection:** The system pastes this fresh text into the LLM's Context Window (hidden from you).
6. **The Final Prediction:** Now the LLM starts predicting again.
 - *Context:* User asked for score. + [Hidden Search Result: Ind is 240/3]
 - *Model Predicts:* "The current score is India 240 for 3."

Analogy:

- The LLM is a smart professor locked in a windowless room (Frozen Brain).
- The Internet Tool is a runner who can leave the room.
- When you ask about the weather, the Professor writes a note: "*Check the weather.*"
- The runner goes out, checks, comes back, and shows the Professor a piece of paper.
- The Professor reads the paper and tells you the answer.

BreakDown of step 4 in the above process -

The "App" (the software code) goes to the internet. The "Model" (the AI brain) stays inside the server and waits.

Think of "ChatGPT" not as one thing, but as **two separate parts** working together.

Here is the breakdown for your notes:

The Brain vs. The Body

1. The Distinction

- **The Model (e.g., GPT-4):** This is the **Brain**. It is a mathematical file. It sits on a server. It can *only* read text and write text. It has **no internet connection**. It cannot open Google. It cannot "see" the web.
- **The App (The "System"):** This is the **Body/Software**. This is the website code (Python/JavaScript) that OpenAI wrote. It handles your login, your chat history, and it **has the internet connection**.

Question:- What is the difference between baseLLM and AI agent like gpt or gemini isn't gpt or gemini is the LLM itself?



The Final Architecture: Models vs. Apps

1. The Definitions

- **The Base LLM (The Engine/Brain):**
 - *Examples:* GPT-4, GPT-4o, GPT-5, Gemini 1.5 Pro, Gemini 2.0 Flash, Claude 3.5 Sonnet.
 - *Role:* These are the static mathematical files. They are smart, but they are **blind, deaf, and paralyzed**. They cannot see the internet, remember your name, or run code on their own. They only process text.
- **The App (The Interface/Body):**
 - *Examples:* ChatGPT, Gemini (the website), Microsoft Copilot.
 - *Role:* These are the software wrappers. They provide the "body" that gives the brain access to the outside world.

2. The "Access to Tools" Relationship

You asked: *"Is the app the thing that has access to tools?"*

Yes.

3. The Grand Map

Component	OpenAI Ecosystem	Google Ecosystem	Anthropic Ecosystem
The App (The Car) (Has internet, tools, memory)	ChatGPT	Gemini (The Website)	Claude.ai
The Base LLM (The Engine) (Just the intelligence)	GPT-4o GPT-5 (Future)	Gemini 1.5 Pro Gemini 2.0 Flash	Claude 3.5 Sonnet

Final Conclusion

Equation of Modern AI:

Useful AI=Base LLM (Brain)+App Wrapper (Tools/Memory)

- **GPT-4o** creates the answer.
- **ChatGPT** delivers the answer and manages the tools (Web Search, Canvas, Voice Mode) that make the answer possible.

Question:- Why the Latest Version is Smarter

Feature	Old Version (e.g., GPT-3)	New Version (e.g., GPT-4o)
Resolution	Low (Fewer Parameters). Misses subtle patterns.	High (More Parameters). Captures complex rules.
Data Source	Internet Chatter (Conversation).	Code & Textbooks (Logic & Structure).
Training Style	Question -> Answer	Question -> Steps -> Answer
Math Strategy	Tries to memorize/guess the result.	Tries to simulate the algorithm of calculation.

Analogy:

The old model was a student who tried to memorize the answer key to the test.

The new model is a student who actually practiced the formulas and studied the textbook.

Question:- How do Multimodal Models work? Do they convert images and video into tokens the same way they convert text, or do they use a completely different process?"

The Answer (The "Universal Language" of Math)

The Short Answer: YES. To an AI, an image, a video, a sound, and a word are all exactly the same thing: **Lists of Numbers**.

The model does not have "eyes" to see images or "ears" to hear audio. It uses a process called **Tokenization** to turn *everything* into a sequence of numbers.

Here is how the model "tokenizes" reality.

1. Text Tokens (Recap)

We know this one. We break a sentence into chunks.

- **Input:** "Cat"
- **Math:** [ID: 4522]

2. Image Tokens (The "Puzzle" Method)

How do you turn a picture into a list of numbers like a sentence?

- **The Method:** Patching (Vision Transformers).
- **The Process:**
 1. Imagine a photo of a cat.

2. The model cuts the photo into a grid of tiny squares (e.g., pixels).
3. 16×1616×16
4. Each tiny square (patch) is flattened into a row of numbers representing the color values.
5. **The Result:** The image is now just a sequence of "Visual Tokens."
- **The Prediction:** The model predicts the next "patch" of the image just like it predicts the next word of a sentence.
 - *Sequence:* [Top_Left_Corner_Patch] [Next_Patch] [Next_Patch].

Analogy: Think of it like "Paint by Numbers." The AI predicts which number (color/shape) goes in the next square of the grid.

3. Audio Tokens (The "Spectrogram" Method)

How do you turn sound into numbers?

- **The Method: Spectrograms.**
- **The Process:**
 1. Sound is a wave. The computer turns this wave into a visual chart called a *Spectrogram* (Frequency vs. Time).
 2. The model slices this chart into tiny time-steps.
 3. Each time-step becomes a token.
- **The Prediction:** The model predicts the next millisecond of sound.

4. Video Tokens (The "Stack" Method)

- **The Process:** Video is just a stack of Images (Frames) + Audio.
- **The Tokens:** The model treats video as a long sequence of Image Tokens followed by Audio Tokens.

The "Unified Vocabulary" (The Modern Breakthrough)

In the past, we had one AI for text and a different AI for images.

Now (GPT-4o / Gemini 1.5 Pro): They are **Native Multimodal Models**.

This means they have a **Giant Dictionary** that contains everything mixed together.

The Model's Vocabulary looks like this:

1. Word: "Apple" (ID: 101)
2. Word: "Run" (ID: 102)
3. Image Patch: "Blue Sky Corner" (ID: 50001)
4. Image Patch: "Cat Ear" (ID: 50002)
5. Sound: "High C Note" (ID: 80001)

How the Model Generates a Video:

It outputs a stream of IDs:

[50001] [50002] [80001] ...

(Image Part)

(Image Part)

(Sound Part) ...

Then, the **Detokenizer** takes these numbers and assembles them into pixels and sound waves instead of text letters.

Input Type	How it becomes a Token	What the AI sees
Text	Split words into chunks.	[83, 12, 992]
Image	Cut image into tiny square patches.	[Patch_1, Patch_2, Patch_3]
Audio	Slice sound waves into time steps.	[Wave_1, Wave_2, Wave_3]
Video	Stack of Image Patches + Audio tokens.	[Frame1_Patch, Frame2_Patch...]

Conclusion

"It's all just Math."

The Transformer model (the brain) doesn't care if the input is Shakespeare or a picture of a dog. As long as we can turn the data into a **sequence of numbers**, the model can predict the next number.

- Predicting the next number for **Text** = Writing.
- Predicting the next number for **Pixels** = Drawing.

An extended addition to the above problem/question -

How AI "Reads" a PDF (The Extraction Process)

1. The "Envelope" Analogy

Think of a PDF file like a **Sealed Envelope**.

- **The Model (Brain):** Cannot touch or open envelopes. It only accepts loose sheets of paper (Tokens).
- **The System (App):** Must open the envelope, take out the letter, and hand the text to the Model.

2. Method A: The Text Extraction Route (Most Common)

This is used for standard documents (resumes, contracts, essays).

1. **Upload:** You drop the PDF into the chat.
2. **The App (Parsing):** The system runs a script (like a Python tool) that strips away the PDF formatting and pulls out the raw strings of text.
 - *PDF:* "Chapter 1" (Bold, Size 14 font)
 - *Extracted Text:* "Chapter 1" (Plain text)
3. **The Injection:** The system secretly pastes this text into your chat history.
 - *Hidden Prompt:* [System: The user uploaded a file. Here is the content: "Chapter 1..."]
4. **The Tokenization:** The model converts that plain text into IDs ([832, 11, 99...]).
5. **The Prediction:** The model answers your questions based on those tokens.

Limitation: Since this method strips formatting, the model might lose track of complex tables or layout. It just sees a stream of words.

3. Method B: The Vision Route (For Charts & Graphs)

This is used by modern models (GPT-4o, Gemini 1.5 Pro) when the PDF contains images, charts, or handwriting.

1. **Conversion:** The App converts the PDF pages into **Images** (screenshots).
2. **Patching:** It uses the **Multimodal** method we just discussed (cutting the image into tiny squares/patches).
3. **Processing:** The model "looks" at the visual structure of the page.
 - *Why this is better:* It can see that "Column A" aligns with "Row B" in a table, or it can read a graph, which "Text Extraction" would miss.

4. What if the PDF is HUGE? (RAG - Retrieval Augmented Generation)

This is crucial for your notes.

If you upload a 500-page book, it might be too many tokens for the Context Window (or too expensive).

The system uses a trick called **RAG (Search)**.

1. **Indexing:** The App chops your PDF into 1,000 small chunks.
2. **Storage:** It saves these chunks in a temporary database.
3. **The Question:** You ask: *"What does the book say about elephant diet?"*
4. **The Search:** The App searches the database for chunks containing "elephant" and "diet."
5. **The Injection:** It **only** pastes those specific pages into the Model's context window.
6. **The Answer:** The model answers based on those specific pages.

PDF Reading = Text Extraction + Context Injection

The AI does not "read" the file format. The System extracts the text (or converts pages to images), turns them into tokens, and pastes them into the beginning of the conversation.

To the Model, there is no difference between **"Text you typed"** and **"Text extracted from a PDF."** It is all just context to predict from.